AIM

To implement a custom Ping program using raw ICMP (Internet Control Message Protocol) sockets in C.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Initialize a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_ICMP.
2. **Resolve Host IP:** Convert the domain name or IPv4 string into a valid target IP address using DNS lookup.
3. **Construct ICMP Packet:**
    - Set ICMP type to ICMP_ECHO (Type 8, Code 0).
    - Set Sequence Number and Process Identifier (PID) fields.
    - Calculate and set the Internet Checksum over the entire ICMP header and payload.
4. **Send Echo Request:** Transmit the packet to the destination address using sendto(). Record the transmission timestamp.
5. **Receive Echo Reply:**
    - Wait for incoming packets with recvfrom() within a timeout window.
    - Parse the outer IP header to extract the inner ICMP packet.

    - Verify that the received packet is an ICMP_ECHOREPLY (Type 0) matching the process ID and sequence number.
6. **Calculate Metrics:** Subtract transmission timestamp from receive timestamp to obtain Round Trip Time (RTT).
7. **Display Output:** Print sequence number, TTL (Time to Live), packet size, and RTT. Repeat for desired number of pings.

Code Implementation (ping.c)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/socket.h>

#include <netinet/in.h>

#include <netinet/ip.h>

#include <netinet/ip_icmp.h>

#include <arpa/inet.h>

#include <netdb.h>

#include <sys/time.h>


#define PACKET_SIZE 64

#define PORT_NO 0

#define TIMEOUT 1 // Timeout in seconds
```

```
// Standard Internet Checksum Algorithm
unsigned short calculate_checksum(unsigned short *buffer, int length) {
    unsigned long sum = 0;
    while (length > 1) {
        sum += *buffer++;
        length -= 2;
    }
    if (length == 1) {
        sum += *(unsigned char *)buffer;
    }
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: sudo %s <hostname/IP>\n", argv[0]);
        return 1;
    }

    int sockfd;
```

```
    struct sockaddr_in dest_addr;
    struct hostent *host;
    char packet[PACKET_SIZE];
    struct icmp *icmp_hdr;
    struct timeval timeout, tv_start, tv_end;

    // 1. Resolve Hostname
    host = gethostbyname(argv[1]);
    if (host == NULL) {
        perror("Could not resolve hostname");
        return 1;
    }

    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_addr = *((struct in_addr *)host-
>h_addr_list[0]);

    // 2. Create Raw Socket for ICMP
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sockfd < 0) {
        perror("Socket creation failed (run as root)");
        return 1;
    }
```

```
    // Set Receive Timeout
    timeout.tv_sec = TIMEOUT;
    timeout.tv_usec = 0;
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, (const
char*)&timeout, sizeof(timeout));


    printf("PING %s (%s) %d bytes of data.\n", argv[1],
inet_ntoa(dest_addr.sin_addr), PACKET_SIZE);


    for (int seq = 1; seq <= 4; seq++) {
        memset(packet, 0, PACKET_SIZE);


        // 3. Construct ICMP Header
        icmp_hdr = (struct icmp *)packet;
        icmp_hdr->icmp_type = ICMP_ECHO;
        icmp_hdr->icmp_code = 0;
        icmp_hdr->icmp_id = getpid();
        icmp_hdr->icmp_seq = seq;
        icmp_hdr->icmp_cksum = 0;


        // Fill remaining bytes with sample data payload
        for (int i = sizeof(struct icmp); i < PACKET_SIZE; i++) {
            packet[i] = i + '0';
```

```
        }

        icmp_hdr->icmp_cksum = calculate_checksum((unsigned
short *)packet, PACKET_SIZE);

        // 4. Send Packet & Record Time
        gettimeofday(&tv_start, NULL);
        if (sendto(sockfd, packet, PACKET_SIZE, 0, (struct
sockaddr *)&dest_addr, sizeof(dest_addr)) <= 0) {
            perror("Failed to send packet");
            continue;
        }

        // 5. Receive Reply
        char recv_buf[1024];
        struct sockaddr_in from_addr;
        socklen_t from_len = sizeof(from_addr);

        int bytes_rcvd = recvfrom(sockfd, recv_buf,
sizeof(recv_buf), 0, (struct sockaddr *)&from_addr, &from_len);
        gettimeofday(&tv_end, NULL);

        if (bytes_rcvd < 0) {
            printf("Request timed out for icmp_seq %d\n", seq);
        } else {
```

```
            // Extract IP header to inspect TTL and payload
            struct ip *ip_hdr = (struct ip *)recv_buf;
            int ip_hdr_len = ip_hdr->ip_hl * 4;
            struct icmp *reply_icmp = (struct icmp *)(recv_buf +
ip_hdr_len);

            // Calculate RTT in milliseconds
            double rtt = (tv_end.tv_sec - tv_start.tv_sec) * 1000.0 +
                         (tv_end.tv_usec - tv_start.tv_usec) / 1000.0;

            if (reply_icmp->icmp_type == ICMP_ECHOREPLY) {
                printf("%d bytes from %s: icmp_seq=%d ttl=%d
time=%.2f ms\n",
                       bytes_rcvd - ip_hdr_len,
                       inet_ntoa(from_addr.sin_addr),
                       reply_icmp->icmp_seq,
                       ip_hdr->ip_ttl,
                       rtt);
            }
        }
        sleep(1); // Wait 1 second between requests
    }

    close(sockfd);
```

```
    return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./myping 127.0.0.1

Output

Server Terminal:

PING 127.0.0.1 (127.0.0.1) 64 bytes of data.

64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.08 ms

64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.06 ms

64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.07 ms

64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.05 ms